GCP SRE & AUTONOMOUS AI ARCHITECTURE REFERENCE

AI Ops Assistant on Google Cloud Platform

Automated Incident Investigation, Telemetry Correlation & Root Cause Analysis using Vertex AI (Gemini)

Target Cloud: Google Cloud Platform (GCP) **LLM Engine:** Gemini 2.5 Flash on Vertex AI **Infrastructure:** Terraform v1.5+

Runtimes: Python 3.12 (Cloud Run Fn), Debian 12 (GCE) **Generated:** October 2026

1. Executive Summary

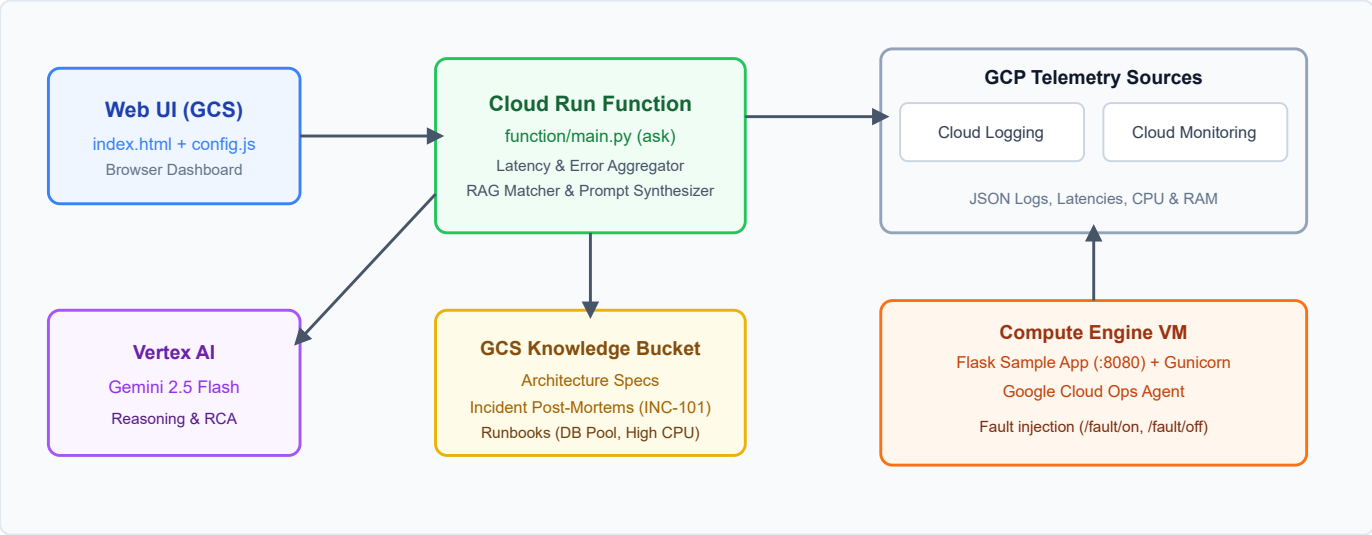
Modern cloud incidents require Site Reliability Engineers (SREs) to rapidly correlate distributed telemetry (logs, error rates, p95/p99 latency spikes, machine health) with institutional knowledge (architecture topologies, post-mortems of previous outages, and standard operating runbooks). In high-stress scenarios, manual correlation takes precious time and is error-prone.

The **AI Ops Assistant on GCP** project demonstrates a complete, autonomous Site Reliability Assistant architecture. It continually gathers live system signals from Google Cloud Logging and Monitoring, pairs them with contextual knowledge files stored in Google Cloud Storage, and submits the combined corpus to Gemini on Vertex AI. The assistant provides an immediate, evidence-grounded diagnosis: pinpointing the root cause, citing exact telemetry proofs, detailing step-by-step triage actions, and recommending surgical remediation steps.

> **Key Innovation:** Grounded In-Context Telemetry Fusion. Rather than asking an LLM generic questions, this system automatically executes targeted Cloud Logging and Cloud Monitoring queries for the incident window, parses latency percentiles and error codes, pulls matching runbooks, and instructs Gemini to reason *strictly* on empirical evidence without hallucinations.

2. System Architecture & Data Flow

The project is partitioned into four primary layers: the monitored workload, the telemetry ingestion pipeline, the serverless AI diagnosis engine, and the web interface.



3. Repository Structure & File Responsibility Map

File / Path	Component	Description & Operational Purpose
`function/main.py`	AI Core	Cloud Run Function backend. Gathers telemetry from Cloud Logging & Monitoring, retrieves knowledge files from GCS, and constructs Gemini prompts to diagnose issues.
`function/requirements.txt`	AI Core	Dependencies: `functions-framework`, `google-cloud-logging`, `google-cloud-monitoring`, `google-cloud-storage`, and `google-genai`.
`sample-app/app.py`	Target App	Flask application with endpoints for `/api/orders`, `/api/checkout`, and fault injection toggles (`/fault/on`, `/fault/off`). Logs structured JSON.
`terraform/main.tf`	IaC	Declares GCP infrastructure: Storage buckets (web, knowledge, build), Compute Engine VM, Cloud Run Function 2nd Gen, IAM roles, and firewall rules.
`terraform/startup.sh.tpl`	IaC	VM bootstrap script: Installs Google Cloud Ops Agent, configures JSON log parsing for `sample_app`, downloads `app.py`, and starts Gunicorn as a systemd service.
`terraform/variables.tf`	IaC	Parameters: `project_id`, `region`, `zone`, `gemini_model` (default: `gemini-2.5-flash`), `allowed_cidr`, and authentication toggles.
`web/index.html`	UI	Single-page diagnosis portal hosted in GCS. Submits queries to the Cloud Run function, displays root causes, and shows raw telemetry evidence.
`knowledge/architecture/sample-app.md`	Knowledge	System topology documentation: Normal latency profiles (orders p95 < 200ms, checkout p95 < 300ms) and database connection pool specifications.
`knowledge/incidents/INC-101.md`	Knowledge	Historical post-mortem: Explains previous checkout timeouts caused by connection pool starvation and describes how it was resolved.
`knowledge/runbooks/db-connection-pool.md`	Knowledge	Step-by-step diagnostic tree and mitigation procedures when DB connection pool exhaustion errors appear.
`knowledge/runbooks/high-cpu.md`	Knowledge	SRE runbook for CPU spikes > 85%, runaway worker threads, and VM scale-up procedures.

4. Core Component Code Analysis

4.1 Monitored Service & Fault Injection (`sample-app/app.py`)

The sample app acts as a realistic target for SRE observability. It measures request durations via Flask hooks and writes JSON logs to stdout:

```
@app.after_request
def _log_request(resp):
    ms = round((time.time() - request._t0) * 1000)
    sev = "ERROR" if resp.status_code >= 500 else "INFO"
    log(sev, f"{request.method} {request.path} -> {resp.status_code}",
        path=request.path, status=resp.status_code, latency_ms=ms)
    return resp
```

When the fault is toggled on (via `/fault/on`), a flag file `/tmp/sample-app-fault` is created so all Gunicorn worker processes observe the failure state simultaneously. The checkout route then injects artificial delay and fails with database exhaustion:

```
@app.route("/api/checkout")
def checkout():
    if os.path.exists(FAULT_FLAG):
        time.sleep(random.uniform(2.5, 5))              # Simulated DB latency
        if random.random() < 0.6:
            log("ERROR", "db connection pool exhausted: timeout acquiring connection (pool_size=10, in_use
=10)",
                component="db", path="/api/checkout")
            return jsonify(error="upstream timeout"), 503
    else:
        time.sleep(random.uniform(0.05, 0.2))
    return jsonify(status="paid")
```

4.2 Telemetry Collector & Diagnostic Engine (`function/main.py`)

The serverless backend handles four distinct stages upon receiving an incident question:

- **Telemetry Aggregation (Cloud Logging):** Gathers up to 1,000 log entries matching `logName="projects/{PROJECT}/logs/sample_app"` over the chosen time window. It compiles latency histograms (p50, p95, max) by route, groups HTTP response codes, and extracts recurring ERROR/WARNING messages.
- **Host Metric Aggregation (Cloud Monitoring):** Gathers mean and peak values for `compute.googleapis.com/instance/cpu/utilization` and `agent.googleapis.com/memory/percent_used` with a 60-second alignment period.
- **Contextual Knowledge Search:** Scans Markdown files in the GCS knowledge bucket, performing set intersection between words in the query/evidence and the runbooks to pick the top-4 relevant documents.
- **Gemini Reasoning Prompt:** The prompt synthesizes the live evidence and runbooks with strict system instructions:

```
SYSTEM = ("You are an SRE assistant. Use ONLY the evidence provided (logs, metrics, latency, runbooks, "
          "architecture, past incidents). Give: 1) Most likely root cause, 2) Evidence that supports it, "
          "3) Numbered diagnostic steps, 4) Suggested fix / mitigation, 5) Confidence and what is unknown.
"
          "If the evidence does not show a problem, say so. Never invent data.")
```

5. Step-by-Step Incident Simulation & Triage Lifecycle

The project provides a fully repeatable feedback loop to test and evaluate the AI agent:

Phase 1: Healthy Baseline Traffic

Send normal requests to populate healthy metrics:

```
APP=$(terraform output -raw app_url)
for i in $(seq 1 30); do
  curl -s $APP/api/orders >/dev/null
  curl -s $APP/api/checkout >/dev/null
done
```

Latency remains < 200ms; status 200; error count is zero.

Phase 2: Problem Injection

Activate the database connection fault:

```
curl $APP/fault/on
for i in $(seq 1 15); do
  curl -s $APP/api/checkout >/dev/null &
done
wait
```

Checkout latency reaches 4–5s; HTTP 503 errors and connection pool errors appear in logs.

Phase 3: AI Diagnosis

In the Web UI, ask:

"Why is checkout slow and returning errors?"

The assistant detects error logs, correlates them with runbook `db-connection-pool.md` and historical `INC-101.md`, and outputs root cause and recovery steps.

Phase 4: Recovery

Disable the fault flag to restore normal state:

```
curl $APP/fault/off
```

Service returns to normal sub-300ms latencies and zero 5xx error responses.

5.1 Sample AI Output Breakdown

1. Most Likely Root Cause: Database connection pool exhaustion on the `/api/checkout` pathway.

2. Supporting Evidence:

- Log entries: `"db connection pool exhausted: timeout acquiring connection (pool_size=10, in_use=10)"` (count: 9).
- Latency: `/api/checkout` p95 latency is 4,820ms (baseline < 300ms), while `/api/orders` remains normal at 110ms.
- Resource Metrics: CPU utilization is low (~4.2%), ruling out VM hardware starvation or runaway CPU processes.

3. Diagnostic Steps:

1. Confirm DB connection pool exhaustion in Cloud Logging.
2. Inspect active database connections, query locks, or hung transactions in the database tier.
3. Review runbook `knowledge/runbooks/db-connection-pool.md` .

4. Mitigation & Fix:

- Temporary: Restart application service on VM (`sudo systemctl restart sample-app`) to drain hung connections.
- Permanent: Increase database connection pool size from 10 to 30, and inspect slow query performance as documented in past post-mortem `INC-101.md` .

5. Confidence: High (95%). All symptoms match documented incident `INC-101` and runbook specifications.

6. Production Readiness & Architecture Roadmap

While the project is fully functional as an architectural demonstration, real-world enterprise deployments should address the following areas:

Area	Demo Implementation	Production Recommendation
Authentication & Security	Cloud Run Function has `allow_unauthenticated = true`; web bucket is public.	Deploy Google Cloud API Gateway with Identity Platform (OIDC/JWT tokens), front the UI with Cloud CDN + Cloud Armor (IAP - Identity-Aware Proxy), and make GCS buckets private.
Knowledge Retrieval (RAG)	Basic in-memory Python keyword set intersection across text files.	Adopt **Vertex AI RAG Engine** or **Vertex AI Search** using vector embeddings to enable semantic retrieval across thousands of pages of Confluence/GitHub documentation.
Telemetry Depth	Cloud Logging text queries & basic VM CPU/memory metrics.	Integrate **Cloud Trace** (distributed tracing with OpenTelemetry) to pinpoint exact database call spans, lock wait times, and third-party API dependencies.
Automated Remediation (Human-in-the-Loop)	Read-only recommendations in text form.	Provide executable action buttons in the UI (e.g., "Restart Service", "Scale Instance Group", "Rollback Release") backed by Cloud Run executing signed IAM-restricted Cloud Tasks.

7. Deployment & Operational Quick Reference

```
# 1. Authenticate to Google Cloud
gcloud auth application-default login

# 2. Initialize and deploy infrastructure via Terraform
cd terraform
terraform init
terraform apply -var="project_id=YOUR_PROJECT_ID" -var="allowed_cidr=YOUR_IP/32"

# 3. Retrieve deployment endpoints
terraform output web_url       # URL for browser dashboard
terraform output function_url  # Cloud Run Function REST endpoint
terraform output app_url       # Direct IP and port for the sample application VM

# 4. Teardown all resources when done
terraform destroy -var="project_id=YOUR_PROJECT_ID" -var="allowed_cidr=YOUR_IP/32"
```